arXiv:2112.11610v2 [cs.CV] 29 Dec 2021

EyePAD++: A Distillation-based approach for joint Eye Authentication and Presentation Attack Detection using Periocular Images

Prithviraj Dhar[1], Amit Kumar[2], Kirsten Kaplan [2], Khushi Gupta[2], Rakesh Ranjan[2], Rama Chellappa[1]
[1]Johns Hopkins University, [2]Reality Labs, Meta
{pdhar1,rchella4}@jhu.edu, {akumar14,kkaplan,khushigupa,rakeshr}@fb.com

Abstract

A practical eye authentication (EA) system targeted for edge devices needs to perform authentication and be robust to presentation attacks, all while remaining compute and latency efficient. However, existing eye-based frameworks a) perform authentication and Presentation Attack Detection (PAD) independently and b) involve significant pre-processing steps to extract the iris region. Here, we introduce a joint framework for EA and PAD using periocular images. While a deep Multitask Learning (MTL) network can perform both the tasks, MTL suffers from the forgetting effect since the training datasets for EA and PAD are disjoint. To overcome this, we propose Eye Authentication with PAD (EyePAD), a distillation-based method that trains a single network for EA and PAD while reducing the effect of forgetting. To further improve the EA performance, we introduce a novel approach called EyePAD++ that includes training an MTL network on both EA and PAD data, while distilling the 'versatility' of the EyePAD network through an additional distillation step. Our proposed methods outperform the SOTA in PAD and obtain near-SOTA performance in eye-to-eye verification, without any pre-processing. We also demonstrate the efficacy of EyePAD and EyePAD++ in user-to-user verification with PAD across network backbones and image quality.

1. Introduction

Eye Authentication (EA) using irises has been widely used for biometric authentication. With the current advancements in head-mounted technology, eye-based authentication is likely to become an essential part of authenticating users against their wearable devices. While highly accurate, EA systems are also vulnerable to 'Presentation Attacks' (PA) [14, 47]. These attacks seek to fool the authentication system by presenting artificial eye images, such as printed iris images of an individual [5, 16], or cosmetic contacts [21]. While researchers have proposed methods to train networks that achieve SOTA performance in either

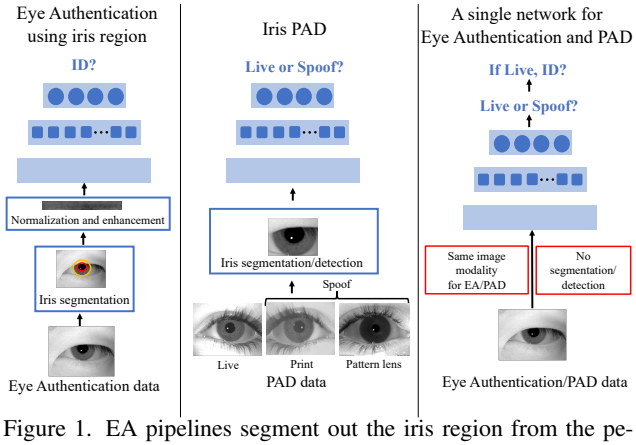


Figure 1. EA pipelines segment out the iris region from the periocular image and normalize the iris image before feeding it to the network. Segmentation/detection is also used in most PAD pipelines. We train a single network for both EA and PAD without any pre-processing steps, using the entire periocular image.

EA or PAD, a practical eye-based biometric system that can be used on edge devices must be able to perform both of these tasks accurately and simultaneously, with low latency and in an energy efficient manner. Therefore, in this work we propose strategies to develop a single deep network for both EA and PAD using periocular images.

One of the key steps in EA is pre-processing. Most EA frameworks [45, 53] use an auxiliary segmentation network to extract the iris region from the periocular image. The iris is then unwrapped into a rectangular image and is fed to the eye authentication system. This geometric normalization step was first proposed in [7]. Pre-processing is also an important step in iris PAD pipelines that requires another segmentation network [14, 47], or a third party iris detection software [38, 41]. Such pre-processing steps potentially make the EA and PAD pipelines computationally expensive, making it impractical to embed these biometric systems on edge devices with limited computational resources. We investigate and propose techniques to perform EA and PAD using the entire periocular image without any active pre-processing. In doing so, we adhere to our goal of using a *single* network in a truer sense.

For a given subject, irises of left and right eyes demonstrate different textural patterns. In most of the existing works in EA [45, 53], the CNN models are trained on the left irises for classification. During evaluation, a right iris is verified against the right irises of the same or other subjects. We refer to this evaluation method as '*eye-to-eye verification*'. However, a more practical protocol would be to perform user-to-user verification, i.e. consider both left and right eyes of a given test subject (query user) and verify it against one or more pairs of left-right irises of same or different user (i.e. gallery user). To this end, we propose a new evaluation protocol to match the the left-right pair of a query user with that of a gallery user.

We consider the problem of EA and PAD as a disjoint multitask learning problem because the authentication task presumes real images, which is why the current datasets for EA do not include PAD labels. A possible single-network solution is to train a deep multitask network for both tasks, alternately training the EA and PAD branches with their respective dataset each iteration (as done in [37]). However, several works [18, 22, 27] have shown that Multitask Learning (MTL) frameworks for disjoint tasks demonstrate forgetting effect (see Sec. 3). Hence, we propose two novel knowledge distillation-based techniques called EyePAD and EyePAD++ to incrementally learn EA and PAD tasks, while minimizing the forgetting effect. In summary, we make the following contributions in this work:

1. We propose a user-to-user verification protocol that can be used to authenticate one query user against one or many samples of a gallery user. This is more practical than the existing protocol for eye-to-eye verification.

2. To the best of our knowledge, we are the first to explore the problem of EA and PAD using a single network. We introduce a new metric called Overall False Rejection Rate (OFRR) to evaluate the performance of the entire system (EA and PAD), using only authentication data.

3. We propose a distillation-based method called **Eye** Authentication with **P**resentation **A**ttack **D**etection (EyePAD) for jointly performing EA and PAD. To further improve the verification performance, we propose EyePAD++. EyePAD++ inherits the versatility of the EyePAD network through distillation and combines it with the specificity of multitask learning. EyePAD++ consistently outperforms the existing baselines for MTL, in terms of OFRR. We show the efficacy of EyePAD and EyePAD++ across different network backbones (Densenet121 [20], MobilenetV3 [19] and HRnet64 [44]), and image quality degradation (blur and noise). Additionally, we apply our methods to jointly perform eye-to-eye verification and PAD, following the commonly used train-test protocols. Although the current SOTA approaches use pre-processing, our proposed methods outperform the existing SOTA in PAD task, and obtain comparable user-to-user verification performance without any pre-processing.

Method	EA	PAD	Pre-processing
IrisCode [29]	✓	✗	Segmentation, geometric normalization
Ordinal [42]	✓	✗	Segmentation, geometric normalization
UniNet [53]	✓	✗	Segmentation, geometric normalization
DRFnet [45]	✓	✗	Segmentation, geometric normalization
[35]	✗	✓	Segmentation, geometric normalization
[14]	✗	✓	Segmentation, geometric normalization
[33]	✗	✓	Cropping
DensePAD [47]	✗	✓	Segmentation, geometric normalization
[17]	✗	✓	Segmentation with UIST [38]
D-net-PAD [41]	✗	✓	Detection with VeriEye
[3]	✗	✓	Detection with [2]
PBS, A-PBS [11]	✗	✓	None
EyePAD (ours)	✓	✓	None
EyePAD++ (ours)	✓	✓	None

Table 1. Pre-processing steps in recent EA/PAD frameworks

2. Related work

Eye authentication using irises: Daugman [6, 7] introduced the first automated system for EA by applying Gabor Filters to the normalized image for generating spatial barcode-like features (IrisCode). More recently, several works have proposed using deep features for EA. [12] proposed DeepIrisNet, the first deep learning-based framework for generalized EA, followed by [13, 32, 43]. [53] presents UniNet, that consists of two components: one for generating discriminative features (FeatNet) and the other for segmenting the iris and non-iris region (MaskNet). Both of these components accept the normalized iris images that also requires segmentation. [45] uses dilated convolution kernels for training CNNs for EA. [51] presents an encoder-decoder pipeline to extract multi-level iris features and use an attention module to combine the multi-level features.

Eye-based Presentation Attack Detection: PAD in periocular images has received significant attention from the deep learning community in the past few years [3, 11, 31, 33, 41, 47]. [25, 46] propose fusing handcrafted and CNN features to detect PA. [10] fuses the features from different layers in a deep network extracted for normalized iris images for PAD. [41, 47] show that DenseNet architecture helps to achieve high PAD accuracy. [17] proposes dividing the iris region into overlapping patches and training CNNs using these patches. [3] introduces an attention guided mechanism to improve PAD accuracy. [11] introduces a binary pixel-wise supervision with self attention to help the network to find patch-based cues and achieve high performance in PAD.

All of the EA algorithms use the normalization process proposed in [7] that requires iris segmentation. Similarly, most PAD algorithms also use auxiliary pre-processing steps such as iris detection/segmentation. A brief summary of

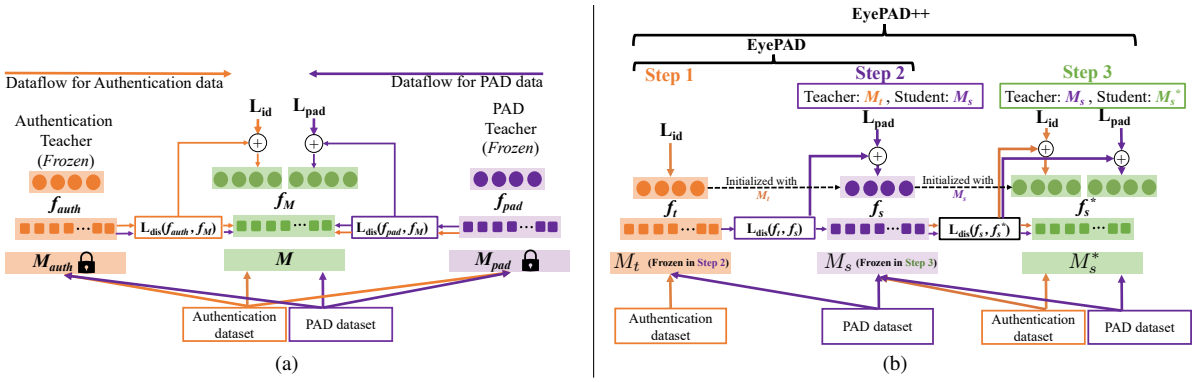


Figure 2. **(a) Baseline**: Multitask Learning with multi-teacher distillation (MTMT) [26] **(b)** Proposed approach **Step 1**: We train M_t for EA, **Step 2 (EyePAD)**: We initialize M_s using M_t and train it for PAD, while distilling EA information from M_t. (c) **Step 3 (EyePAD++)**: We initialize an MTL network M_s^* with the trained M_s and train it to perform both EA and PAD, while distilling the 'versatility' of M_s. EyePAD++ outperforms the MTMT [26] baseline in jointly performing EA and PAD in most of the problem settings.

the preprocessing steps in EA and PAD is given in Table 1.

Disjoint Multitask Learning and Knowledge Distillation: Disjoint multitask learning (MTL) is the process of training a network to perform multiple tasks using data samples that have labels for either of the tasks, but not for all the tasks. Training a single network for EA and PAD is a disjoint MTL task because EA datasets do not include PAD labels. One solution is to follow the existing disjoint multitask learning strategies [24, 28, 37] and update each branch of the network alternately. However, it is well known [22, 27] that alternating training suffers from the forgetting effect [27] and degrades performance in multitask learning. Knowledge Distillation (KD) [15] has been commonly used to reduce forgetting in continual learning [9, 27, 39, 40, 52]. Inspired by this, [22, 26] employ feature-level KD for multitasking. In [22], KD is used to distill the information from the network from a previous iteration $i-1$ that was updated for task A (teacher), while training it to perform task B in the current iteration i (student). However, in this scenario, the teacher network is not fully trained in the initial few iterations and thus the distillation step may not help preserve task A information. Similar to [22], we propose strategies employing feature-level KD for disjoint multitasking (EA and PAD). But, unlike [22], we ensure that the teacher network in our proposed methods is fully trained in one or more tasks.

3. Proposed approach

Our objective is to build a single network that is proficient in performing two disjoint tasks: EA and PAD. We intend to build this framework for edge devices with limited on-device compute. Thus, we exclude any pre-processing step for detecting or segmenting the iris region and use the entire periocular image as input. Mutitask Learning (MTL) is a possible approach in this scenario. Most of the MTL methods for disjoint tasks [37] alternately feed the data from different tasks. However, as shown in [22], MTL demonstrates the forgetting effect. Consider an MTL network with shared backbone and different heads designed to perform two tasks A and B. Suppose that the training batches for task A and B are fed to this MTL network alternately. Here, the weights of the shared backbone modified by the gradients corresponding to the loss for task A in iteration i, may be rewritten in the next iteration $(i+1)$ by the gradients corresponding to the loss for task B. This may lead to forgetting of task A.Therefore, instead of MTL, we propose to use knowledge distillation to learn both tasks through a single network. Here, we intend to first train a teacher network M_t for EA, following which we train a student network M_s for PAD, while distilling the authentication information from M_t to M_s to minimize the forgetting effect.

3.1. Eye Authentication with Presentation Attack Detection (EyePAD) and EyePAD++

We now explain the steps in our proposed methods: EyePAD and EyePAD++ (Fig. 2b):

Step 1: We train the teacher network M_t using periocular images from the EA dataset to perform EA. Similar to [45], we use triplet loss to train M_t. We first extract features f_i for all the images using the penultimate layer of M_t. To select the nth triplet in a given batch, we randomly select an anchor feature $f_a^{(n)}$ belonging to category C . After that, we select the hardest positive feature $f_{pos}^{(n)}$ and the hardest negative feature $f_{neg}^{(n)}$ as follows:

$$f_{pos}^{(n)} = \underset{i \in C, i \neq a}{\operatorname{argmax}}(\|f_i^{(n)} - f_a^{(n)}\|^2), \; f_{neg}^{(n)} = \underset{i \notin C}{\operatorname{argmin}}(\|f_i^{(n)} - f_a^{(n)}\|^2)$$

Then we compute the triplet loss L_{id} for the entire batch (of size N) as:

$$L_{id} = \frac{1}{N} \sum_{n=1}^{n=N} \max(\|f_{pos}^{(n)} - f_a^{(n)}\|^2 - \|f_{neg}^{(n)} - f_a^{(n)}\|^2 + \alpha, 0) \quad (1)$$

where α denotes the distance margin.

Step 2 (Feature-level knowledge distillation - EyePAD): We initialize a student network M_s using M_t, and train it for PAD. Let I be an image from the PAD dataset. I is fed to

both M_t and M_s, to obtain features f_t and f_s, extracted using the penultimate layer of the corresponding networks. To constrain M_s to process an eye image like M_t, we employ feature-level KD and minimize the cosine distance between f_t and f_s using the proposed distillation loss L_{dis}.

$$L_{dis}(f_s, f_t) = 1 - \frac{f_s \cdot f_t}{\|f_s\|\|f_t\|}. \quad (2)$$

Our application of feature-level KD is inspired by [40]. Note that we do not apply KD on the output scores as done in [27], since eye-based matching protocols like [45] use the features from the penultimate layer (and not the output score vector). Additionally, we would like M_s to classify a given image as live (also referred to as 'real' or 'bona-fide') or spoof using L_{pad}, which is a standard cross-entropy classification loss. Combining these constraints, we train M_s using the multitask classification loss L_{multi} as

$$L_{multi} = L_{pad} + \lambda_1 L_{dis}, \quad (3)$$

where λ_1 is used to weight L_{dis}. In this step, the teacher M_t remains frozen. To evaluate the verification performance, we use features extracted from the penultimate layer of the trained M_s for the test EA data and perform user-to-user verification. We feed the test PAD data to M_s and evaluate its performance in live/spoof classification. We find that the student network M_s obtained from EyePAD is a versatile network that is effective for both EA and PAD. However, compared to M_t (that was only trained for EA), M_s obtains slightly lower verification performance. We hypothesize that M_s demonstrates this drop in performance because it was never trained for EA. Therefore, we introduce an additional step to train an MTL network (initialized with M_s) while distilling the versatility of the EyePAD student to this network (Fig. 2b).

Step 3 (EyePAD++): We initialize a new student network M_s^* using M_s. M_s^* is trained for both EA and PAD in an MTL fashion. Following the commonly used strategies for disjoint multitasking [37], the batches from EA and PAD data are alternated after every iteration. To reduce forgetting, we additionally constrain M_s^* to mimic M_s, which acts as its teacher, using the same knowledge distillation used in step 2. We feed the training image to both M_s and M_s^* and obtain features f_s and f_s^* respectively. M_s remains frozen in this step. We use them to compute L_{dis} as:

$$L_{dis}(f_s, f_s^*) = 1 - \frac{f_s \cdot f_s^*}{\|f_s\|\|f_s^*\|}. \quad (4)$$

When authentication data is fed to M_s^* (say, during iteration i), we compute L_{multi}^{id} as follows:

$$L_{multi}^{id} = L_{id} + \lambda_2 L_{dis} \quad (5)$$

Here, L_{id} is the triplet loss from Eq. 1. When PAD data is fed to M_s^* (during iteration $i+1$), we compute L_{multi}^{pad} as:

$$L_{multi}^{pad} = L_{pad} + \lambda_2 L_{dis} \quad (6)$$

(a) (b) (c)

Figure 3. We perform train and test our networks on (a) the original (clean) datasets, (b) their blurred and (c) their noisy versions.

where L_{pad} is a standard classification loss used in step 2 of EyePAD. Thus, L_{multi}^{id} and L_{multi}^{pad} are alternately used to optimize M_s^*. For inference, we use the trained M_s^* for user-to-user verification and PAD. Hyperparameter details for EyePAD and EyePAD++ are provided in the supplementary material.

4. Experiments

4.1. Baseline methods

Single task networks: To estimate the standard user-to-user verification (EA) and PAD performance, we test the 'EA only' and 'PAD only' networks. The 'EA only' network is the teacher network M_t used in Step 1 of EyePAD.

Multitask Learning (MTL): We train a multitask network for EA and PAD by alternately feeding EA and PAD batches (see step 3 of EyePAD++) and alternately optimizing using L_{pad} (Sec. 3.1) and L_{id} (Eq. 1).

Multi-teacher Multitasking (MTMT) [26]: MTMT [26] is a recently proposed multitask framework that combines MTL with multi-teacher knowledge distillation. MTMT has been shown to outperform MTL and other SOTA multitask methods such as GradNorm [4]. Here, single task networks are first trained in specific tasks. An MTL network M is then trained for multiple tasks, while information from the single task networks is distilled into M. A key difference between MTMT and EyePAD++ is that MTMT enforces distillation from multiple task-specific teachers whereas EyePAD++ includes distillation from a single teacher that is proficient in performing multiple tasks. We implement MTMT as one of our baselines for joint EA and PAD (Fig. 2a). Firstly, we train two single task models: M_{auth} for EA and M_{pad} for PAD, and then distill information from them while training a student MTL network M. We use the same feature-level distillation used in EyePAD (Step 2) and EyePAD++. A given image is fed to M_{auth}, M, and M_{pad}, generating features from the penultimate layers f_{auth}, f_M and f_{pad}, respectively. L_{dis} then constrains f_M to be closer to f_{auth} and f_{pad}. M_{auth} and M_{pad} remain frozen in this step. We alternately feed the training batches for EA and PAD. So, when EA data is forwarded to M_{auth}, M_{pad}, M, we optimize M using L_{mtmt}^{id}.

$$L_{mtmt}^{id} = L_{id} + \lambda_{auth} L_{dis}(f_{auth}, f_M) + \lambda_{pad} L_{dis}(f_{pad}, f_M) \quad (7)$$

Here L_{id} is the triplet loss defined in Eq.1. $\lambda_{auth}, \lambda_{pad}$ denote the distillation weights from teacher M_{auth} and M_{pad},

	User-to-user verification (EA)		PAD	
	Data	# images	Data	# images
Train	206 users from ND-Iris-0405	7949	Train split of CU-LivDet (2013,2015,2017), ND-LivDet (2013,2015,2017)	14600
Test	150 users from ND-Iris-0405	4231 (2925 query, 1306 gallery)	Test split of CU-LivDet (2013,2015,2017)	7532

Table 2. Statistics for datasets used for EA with PAD

respectively. Similarly, when PAD data is forwarded, we optimize M using L^{pad}_{mtmt}.

$$L^{pad}_{mtmt} = L_{pad} + \lambda_{auth} L_{dis}(f_{auth}, f_M) + \lambda_{pad} L_{dis}(f_{pad}, f_M) \quad (8)$$

Here L_{pad} is standard classification loss for live/spoof classification. We provide the hyperparameter information for MTMT [26] in the supplementary material.

4.2. Datasets and network architectures used

We summarize the datasets used in our work in Table 2.

EA dataset: We use the ND-Iris-0405 [1, 34] dataset, used widely for eye authentication using irises. The dataset consists of 356 users that are divided into two subsets: U_{train} (randomly selected 206 users) and U_{test} (remaining 150 users). Using the distinct left and right eye images for the users in U_{train} gives us 412 (206×2) categories, which we use to train models for EA. For a given user u in U_{test}, we select 10 left and 10 right eye images to build the query set q_u. Similarly, we select 5 left and 5 right eye images to build the gallery set g_u for user u. Repeating this for all the users in U_{test}, we obtain the query set $Q = \{q_u, \forall u \in U_{test}\}$ and gallery set $G = \{g_u, \forall u \in U_{test}\}$. To enable other researchers replicate our experiments, we provide the train and test splits in the supplementary material.

PAD dataset: For PAD training data, we combine the official training splits of CU-LivDet and ND-LivDet from the LivDet challenges in 2013[49], 2015[50], and 2017[48]. We build the PAD test dataset by combining the official test splits of the CU LivDet dataset from the 2013, 2015 and 2017 challenges. CU-LivDet consists of three categories: Live, patterned lens and printed images. ND-LivDet consists of two categories: Live and patterned lens.

Image quality degradation: The datasets we use in this work are academic datasets [1, 48–50] with high quality images (Fig. 3a). However, real-world authentication on edge devices rely on small sensors which capture low-resolution images. Also, environmental conditions like lighting may further degrade the image quality. Therefore, in addition to using the original datasets, we also perform experiments by degrading the datasets (separately): (i) Blur: We add Gaussian blur with a random kernel size between 1 and 5 to the training images, and add blur with kernel size of 5 to the test images (Fig. 3b). (ii) Noise: We add Additive White Gaussian Noise with a standard deviation $\sigma = 3.0$ (Fig. 3c).

Protocol 1 User to User verification (1 Query, K Gallery)

1: **Required**: Model M, Query dataset Q, Gallery dataset G
2: **Initialize**: Similarity dictionary S=[]
3: **Initialize**: Left and right Query dictionary Q_L, Q_R
4: **for** Query user q_A in Q and Gallery user g_B in G **do**
5: Left query $q_A^{(L)} \leftarrow$ RandomSelect(q_A, 1, Left)
6: Right query $q_A^{(R)} \leftarrow$ RandomSelect(q_A, 1, Right)
7: $g^{(L)}_{B,1}, g^{(L)}_{B,2} \dots g^{(L)}_{B,K} \leftarrow$ RandomSelect(g_B, K, Left)
8: $g^{(R)}_{B,1}, g^{(R)}_{B,2} \dots g^{(R)}_{B,K} \leftarrow$ RandomSelect(g_B, K, Right)
9: $Q_L[q_A] = q_A^{(L)}, Q_R[q_A] = q_A^{(R)}$
10: Left query feature $f^{(L)}_{q_A} = M(q_A^{(L)})$
11: Right query feature $f^{(R)}_{q_A} = M(q_A^{(R)})$
12: Left and right gallery features $f^{(L)}_{g_B}, f^{(R)}_{g_B} = \frac{1}{K}\sum_{k=1}^{k=K} M(g^{(L)}_{B,k}), \frac{1}{K}\sum_{k=1}^{k=K} M(g^{(R)}_{B,k})$
13: Compute similarity $s(q_A, g_B) = \frac{1}{2}(\text{Similarity}(f^{(L)}_{q_A}, f^{(L)}_{g_B}) + \text{Similarity}(f^{(R)}_{q_A}, f^{(R)}_{g_B}))$
14: $S[q_A, g_B] \longleftarrow s(q_A, g_B)$
15: **end for**
16: TAR, FAR, threshold = ROC(S, EA Ground Truth)
17: t_{auth} = threshold at FAR=10^{-3}

Networks used: We implement our proposed methods and baselines using the Densenet121 backbone [20]. This is motivated by this architecture repeatedly demonstrating high PAD performance [11, 41, 47]. To demonstrate the generalizability of EyePAD and EyePAD++, we repeat our experiments using the HRnet64 [44] and MobilenetV3 [19].

4.3. User to user verification protocol

Most experiments in EA [45, 53] train the model on the left irises of all the users and evaluate them in terms of the eye-to-eye verification accuracy for the right irises. However, in a real-world authentication system, the gallery will most likely have both left and right eye images (instead of only right eye images) for an authorized user, and thus both left and right query images can be used for verification. Moreover, it is more practical to authenticate a user using both eyes, as opposed to only the right eye. Hence, we propose matching one pair of left-right eyes (query) to K pairs of left-right eyes (gallery). We provide the detailed user-to-user verification protocol in Protocol 1. To match query user A (q_A) and gallery user B (g_B), we first randomly select one left eye and one right eye image from q_A. Then, we select K left eye and K right eye images from g_B. After that, we feed the left and right query images to a model M and compute their respective features $f^{(L)}_{q_A}, f^{(R)}_{q_A}$. For gallery user B, we compute the features for the K left eye images using M and average them to compute a single feature $f^{(L)}_{g_B}$ (Line 12 of Protocol 1). In the same way, we compute the average feature $f^{(R)}_{g_B}$ by for the right eye gallery

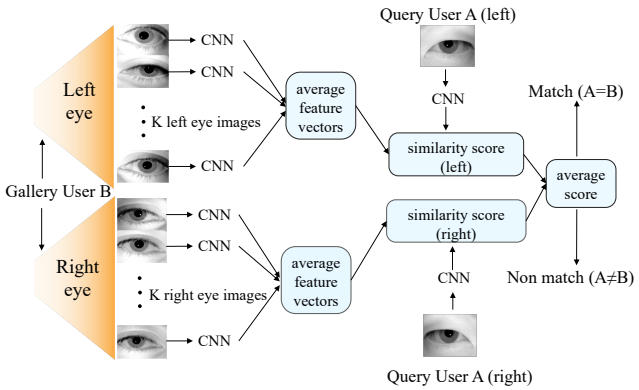


Figure 4. User-to-user verification: Verifying query user A (q_A) against gallery user B (g_B) with K pairs for gallery user B.

Protocol 2 Computing OFRR

1: **Required**: Model M, Query dataset Q, Gallery dataset G
2: **Required**: Similarity dictionary S from Protocol 1
3: **Required**: Query dictionaries Q_L, Q_R from Protocol 1
4: **Required**: Similarity threshold t_{auth} from Protocol 1
5: **Required**: PAD threshold t_{pad} for SAR=5%
6: **Initialize**: Spoof rejects $X_{spoof} = 0$, EA rejects $X_{auth} = 0$
7: **for** Query user q_A, gallery user g_A in Q, G **do**
8: $q_A^{(L)} \leftarrow Q_L[q_A]$, $q_A^{(R)} \leftarrow Q_R[q_A]$
9: Left query PAD logit $o_{q_A}^{(L)} = M(q_A^{(L)})$
10: Right query PAD logit $o_{q_A}^{(R)} = M(q_A^{(R)})$
11: **if** $o_{q_A}^{(L)} > t_{pad}$ or $o_{q_A}^{(R)} > t_{pad}$ **then**
12: $X_{spoof} = X_{spoof} + 1$ // falsely rejected as spoof
13: **else**
14: **if** $S[q_A, g_A] < t_{auth}$ **then**
15: $X_{auth} = X_{auth} + 1$ // falsely rejected as non-match
16: **end if**
17: **end if**
18: **end for**
19: Overall false rejection rate OFRR = $(X_{spoof} + X_{auth})/|Q|$

image. We then compute the similarity between query user A and gallery user B as :

$$s(q_A, g_B) = \frac{1}{2}\left(\frac{f_{g_B}^{(L)} \cdot f_{q_A}^{(L)}}{\|f_{g_B}^{(L)}\|\|f_{q_A}^{(L)}\|} + \frac{f_{g_B}^{(R)} \cdot f_{q_A}^{(R)}}{\|f_{g_B}^{(R)}\|\|f_{q_A}^{(R)}\|}\right) \quad (9)$$

Based on the similarity threshold, a match/non-match is predicted (Fig. 4).

4.4. Metrics for EA and PAD

Performing the similarity computation (Eq. 9) for every possible pairs from (Q, G) and varying the similarity threshold for deciding match/non-match, we compute the ROC curve and report the True Acceptance Rates (TARs) at FAR=$10^{-4}, 10^{-3}, 10^{-2}$. In the biometrics literature [8, 30, 36], it is common to use several gallery samples in authentication. But, for authentication on edge devices, the number of gallery samples that can be used depends on the storage capacity of the edge device. Therefore, for evaluating the EA performance of a given model, we use one query left-right pair and $K = 1/2/5$ gallery left-right pair(s) for verification.

PAD performance is evaluated with four commonly used metrics: (i) True Detection Rate (TDR) at a False Detection Rate of 0.002, (ii) Attack Presentation Classifier Error Rate (APCER), that is the fraction of spoof samples misclassified as Live, (iii) Bonafide Presentation Classifier Error Rate (BPCER), that is the fraction of live samples misclassified as spoof, (iv) Half Total Error Rate (HTER), the average of APCER and BPCER. Following the protocol in [48], we use a threshold of 0.5 for computing APCER and BPCER.

While these metrics gauge either the EA or PAD performance, they cannot jointly evaluate PAD and EA. Hence, we define a new metric in the next subsection.

4.4.1 Overall False Reject Rate (OFRR)

We evaluate EA performance on the test EA data (ND-Iris-0405) and the PAD performance on the test subset of CU-LivDet datasets from the 2013, 2015 and 2017 challenges. An ideal metric must measure PAD and EA performance simultaneously on a single dataset. Such a metric must measure: *How often does the model reject true users from accessing the system?* A true user in the EA dataset can be falsely rejected as: (1) 'Spoof' by the PAD pipeline, or (2) 'Non-match' by the user-to-user verification pipeline. In this regard, we introduce a new metric called **O**verall **F**alse **R**ejection **R**ate (OFRR) for true query users in the EA test subset. The steps for computing the OFRR of true users are summarized in Protocol 2. To determine OFRR, we must first set thresholds for the rates at which PAD misclassifies spoof as live (i.e. Spoof Acceptance Rate or SAR) and EA falsely accepts non-match pairs (FAR). The PAD threshold t_{pad} is computed as the point where SAR=5% when PAD test data is fed to the model. Similarly, when EA test data is fed to the model, the similarity threshold t_{auth} is computed as the point where the user-to-user verification results in FAR=10^{-3}. After computing t_{pad} and t_{auth}, we feed the EA test data to the model again. We then compute the number of query users falsely rejected as spoof (X_{spoof}) using t_{pad}. Here, we reject a query user if at least one of the associated eye images is classified as spoof (Line 12 in Protocol 2). For those query users classified as live, we verify them for EA against matching gallery users, using our user-to-user verification protocol (Fig 4). We compute the number of query users that are falsely rejected as non-match X_{auth} using t_{auth}. Finally, we compute the Overall False Reject Rate (OFRR) as

$$\text{OFRR} = \frac{X_{spoof} + X_{auth}}{|Q|} \quad (10)$$

$|Q|$ = total number of query users in the EA test dataset, which, in our case, is 150 (Sec. 4.2). Ideally, *an authentication system for EA and PAD must have a lower OFRR.*

	User-to-user verification results on ND-Iris-0405 (EA)												PAD results on CU-LivDet			
	1 Query 1 Gallery				1 Query 2 Gallery				1 Query 5 Gallery							
Method	OFRR(↓)	10^{-4}	10^{-3}	10^{-2}	OFRR(↓)	10^{-4}	10^{-3}	10^{-2}	OFRR(↓)	10^{-4}	10^{-3}	10^{-2}	TDR(↑)	APCER	BPCER	HTER(↓)
EA only	-	0.886	0.958	0.996	-	0.891	0.954	0.994	-	0.921	0.979	0.997	-	-	-	-
PAD only	-	-	-	-	-	-	-	-	-	-	-	-	0.971	0.026	0.003	0.015
MTL	0.100	0.693	0.905	0.988	0.074	0.803	0.943	0.986	0.052	0.850	0.956	0.990	0.950	0.036	0.004	0.020
MTMT [26]	0.087	0.919	0.963	0.992	0.068	0.872	0.950	0.989	0.052	0.816	0.923	0.986	0.945	0.051	0.003	0.027
EyePAD	0.079	0.843	0.926	0.993	**0.046**	0.887	0.961	0.997	0.060	0.922	0.952	0.995	0.947	0.036	0.014	0.025
EyePAD++	**0.072**	0.901	0.952	0.990	0.055	0.906	0.966	0.997	**0.043**	0.929	0.983	0.996	0.951	0.034	0.012	0.023
EA only	-	0.832	0.916	0.979	-	0.867	0.943	0.985	-	0.909	0.966	0.992	-	-	-	-
PAD only	-	-	-	-	-	-	-	-	-	-	-	-	0.844	0.073	0.032	0.053
MTL	0.244	0.745	0.871	0.974	0.209	0.801	0.910	0.989	0.199	0.834	0.930	0.994	0.702	0.136	0.034	0.085
MTMT [26]	0.236	0.753	0.894	0.974	0.213	0.841	0.921	0.986	**0.189**	0.822	0.946	0.993	0.576	0.047	0.095	0.071
EyePAD	0.231	0.757	0.876	0.979	0.204	0.796	0.933	0.974	0.196	0.813	0.949	0.987	0.738	0.129	0.024	0.077
EyePAD++	**0.201**	0.830	0.916	0.988	**0.188**	0.854	0.947	0.986	0.192	0.889	0.944	0.987	0.693	0.137	0.029	0.083
EA only	-	0.760	0.901	0.980	-	0.860	0.929	0.984	-	0.897	0.958	0.992	-	-	-	-
PAD only	-	-	-	-	-	-	-	-	-	-	-	-	0.918	0.063	0.004	0.034
MTL	0.184	0.768	0.891	0.981	0.170	0.819	0.927	0.993	0.152	0.865	0.956	0.993	0.879	0.082	0.011	0.046
MTMT [26]	0.168	0.777	0.891	0.979	0.144	0.851	0.927	0.990	0.105	0.869	0.961	0.993	0.883	0.104	0.009	0.057
EyePAD	0.162	0.718	0.852	0.976	0.128	0.757	0.891	0.984	0.094	0.824	0.922	0.991	0.931	0.058	0.005	0.032
EyePAD++	**0.144**	0.777	0.882	0.979	**0.111**	0.797	0.919	0.983	**0.082**	0.878	0.948	0.991	0.926	0.065	0.007	0.036

Table 3. EA and PAD with Densenet121 trained and evaluated on **(top)** original, **(middle)** blurred, **(bottom)** Noisy data: For user-to-user verification, we report TAR@FAR=$10^{-4}, 10^{-3}, 10^{-2}$. For PAD, we report TDR@FDR=0.002 and APCER, BPCER, HTER. OFRR jointly measures EA and PAD performance on ND-Iris-0405. *EyePAD++ obtains the lowest OFRR*. **Bold**: Best, Underlined: Second best

	EA				PAD	
	1 Query 5 Gallery					
Method	OFRR(↓)	10^{-4}	10^{-3}	10^{-2}	TDR(↑)	HTER(↓)
EA only	-	0.919	0.983	0.996	-	-
PAD only	-	-	-	-	0.962	0.026
MTL	0.113	0.815	0.930	0.977	0.921	0.029
MTMT [26]	0.068	0.891	0.945	0.984	0.959	0.017
EyePAD	0.034	0.898	0.968	0.995	0.934	0.029
EyePAD++	**0.031**	0.926	0.976	0.997	0.915	0.031
EA only	-	0.911	0.968	0.992	-	-
PAD only	-	-	-	-	0.801	0.058
MTL	0.293	0.638	0.801	0.924	0.737	0.186
MTMT [26]	0.129	0.904	0.963	0.994	0.646	0.086
EyePAD	0.132	0.902	0.960	0.993	0.766	0.062
EyePAD++	**0.118**	0.915	0.971	0.989	0.655	0.067
EA only	-	0.837	0.952	0.992	-	-
PAD only	-	-	-	-	0.942	0.029
MTL	0.236	0.587	0.787	0.940	0.899	0.045
MTMT [26]	0.114	0.824	0.916	0.975	0.894	0.034
EyePAD	**0.091**	0.800	0.916	0.986	0.914	0.033
EyePAD++	0.093	0.840	0.937	0.989	0.887	0.036

Table 4. EA and PAD with HRnet64, trained and evaluated on the **(top)** original, **(middle)** blurred, **(bottom)** noisy (AWGN σ=3.0) data: For user-to-user verification, we report TAR@FAR=$10^{-4}, 10^{-3}, 10^{-2}$. For PAD, we report TDR@FDR=0.002 and HTER. *EyePAD++ generally obtains the lowest OFRR*. **Bold**: Best, Underlined: Second best. Results with 1 and 2 gallery pairs are provided in the supplementary material.

	EA				PAD	
	1 Query 5 Gallery					
Method	OFRR(↓)	10^{-4}	10^{-3}	10^{-2}	TDR(↑)	HTER(↓)
EA only	-	0.898	0.952	0.995	-	-
PAD only	-	-	-	-	0.925	0.029
MTL	0.110	0.872	0.933	0.985	0.884	0.039
MTMT [26]	0.126	0.859	0.933	0.987	0.793	0.042
EyePAD	0.114	0.887	0.947	0.991	0.859	0.040
EyePAD++	**0.085**	0.901	0.962	0.990	0.883	0.032
EA only	-	0.846	0.921	0.989	-	-
PAD only	-	-	-	-	0.581	0.117
MTL	0.483	0.744	0.855	0.942	0.556	0.128
MTMT [26]	0.464	0.769	0.913	0.963	0.502	0.137
EyePAD	0.447	0.711	0.866	0.956	0.589	0.121
EyePAD++	**0.332**	0.861	0.938	0.983	0.552	0.133
EA only	-	0.817	0.944	0.985	-	-
PAD only	-	-	-	-	0.831	0.041
MTL	0.173	0.761	0.908	0.974	0.762	0.064
MTMT [26]	0.162	0.777	0.906	0.977	0.730	0.059
EyePAD	0.209	0.801	0.927	0.980	0.712	0.065
EyePAD++	**0.137**	0.811	0.912	0.990	0.730	0.080

Table 5. EA and PAD with MobilenetV3, trained and evaluated on the **(top)** original, **(middle)** blurred, **(bottom)** noisy (AWGN σ=3.0) data: For user-to-user verification, we report TAR@FAR=$10^{-4}, 10^{-3}, 10^{-2}$. For PAD, we report TDR@FDR=0.002 and HTER. *EyePAD++ generally obtains the lowest OFRR*. **Bold**: Best, Underlined: Second best. Results with 1 and 2 gallery pairs are provided in the supplementary material.

The user-to user verification and OFRR protocols are run consecutively, and depend on random samples of left and right images of query and gallery users as shown in Protocols 1 (Lines 5,6,7,8) and 2 (Lines 2, 3). Hence, we compute these metrics ten times and report the average.

4.5. Results

EA and PAD with Densenet backbone: We perform the EA and PAD experiments using the Densenet121 backbone for the original datasets and degraded datasets (Table 3). *EyePAD++ obtains the lowest OFRR in most*

of the the problem settings. Moreover, EyePAD++ obtains higher user-to-user verification performance than existing multitasking baselines at most FARs. This demonstrates the advantage of the additional distillation step combined with MTL. The PAD performance demonstrated by EyePAD++ is also comparable to that of other multitasking baselines.

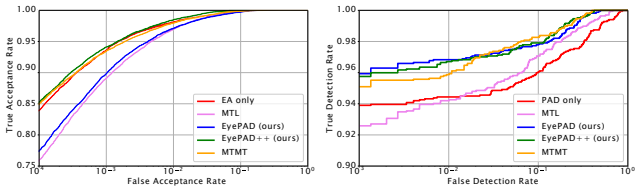
EA and PAD with HRnet64 and MobilenetV3 backbone: To demonstrate the generalizability of our proposed methods, we repeat the same experiments with the HRnet64[44] backbone (Table 4). However, training a Densenet or HRnet64 model is computationally expensive. So, we also perform the same experiment with the MobilenetV3 [19] backbone, that is much more computationally efficient than Densenet (Table 5). More detailed results with 1 or 2 gallery pairs are provided in the supplementary material. Once again we find that *EyePAD++ obtains lower OFRR than MTL and MTMT[26]*. The superiority of EyePAD++ with MobilenetV3 indicates that EyePAD++ can be used for performing EA and PAD on compute engines with low capacity that are available on edge devices.

Eye-to-eye verification with PAD: To compare our proposed methods with current SOTA in PAD and EA, we perform eye-to-eye verification with PAD. Here, for EA, we follow [45, 53] and use the first 25 left eye images of every user in the ND-Iris-0405 dataset [1] for training. We use the first 10 right eye images of the users for testing. For evaluating EA, we use the same eye-to-eye verification in [45, 53]. For PAD, we follow [11, 41] and only use the official train and test split of the CU-LivDet-2017 dataset. We perform this experiment with Densenet121. While training and testing our methods and baselines (Sec. 4.1), we exclude preprocessing. Fig. 5 shows the ROC curves for EA and PAD obtained by all the methods. From Table 6, we infer that *EyePAD and EyePAD++ achieve better PAD performance (i.e. TDR @ FDR=0.002) than the current SOTA PAD algorithms*, without any pre-processing. Moreover, *EyePAD++ achieves higher EA performance (TAR at FAR=10^{-3}) than the comparable baselines* (i.e. EA only network, MTL and MTMT). The EA performance for EyePAD and EyePAD++ is comparable to but slightly lower than that of the SOTA [45, 53], with a difference of less than 4%. We believe that this is difference is due to excluding pre-processing steps for limiting computational cost.

EyePAD++ v/s MTMT [26]: Both EyePAD++ and MTMT combine MTL with feature-level KD. However, the student MTL network in MTMT does not inherit the 'versatility' through distillation since its teachers are single-task models that are not versatile. On the other hand, EyePAD++ uses distillation from a single versatile teacher (M_s), that is proficient in both the tasks. As a result, the student network

	EA		PAD			
Method	TAR(↑)	EER	TDR(↑)	APCER	BPCER	HTER
IrisCode† [29]	0.967	1.88	-	-	-	-
Ordinal† [42]	0.968	1.74	-	-	-	-
UniNet† [53]	0.971	1.40	-	-	-	-
DRFnet† [45]	**0.977**	1.30	-	-	-	-
Winner of [48] †	-	-	-	13.39	0.89	7.10
SpoofNet† [23]	-	-	-	33.00	0.00	16.50
Meta-fusion† [25]	-	-	-	18.66	0.24	9.45
D-net-PAD† [41]	-	-	92.05	5.78	0.94	3.36
PBS [11]	-	-	94.02	8.97	0.0	4.48
A-PBS [11]	-	-	92.35	6.16	0.81	3.48
EA only	0.936	1.48	-	-	-	-
PAD only	-	-	94.02	5.96	0.02	2.99
MTL	0.891	1.88	92.70	9.03	0.00	4.52
MTMT [26]	0.933	1.54	95.51	7.54	0.00	3.77
EyePAD	0.898	1.89	**96.29**	5.68	0.00	2.84
EyePAD++	0.941	1.30	95.99	7.29	0.00	3.65

Table 6. Eye-to-eye verification (TAR@FAR=10^{-3} and Equal Error Rate) and PAD performance (TDR@FDR=0.002). †=Use preprocessing (Segmentation/detection)



(a) Eye-to-eye verification (b) PAD

Figure 5. ROC curves for (a)EA performance on ND-Iris-0405 (b) PAD performance on CU-LivDet

M_s^* in EyePAD++ inherits the versatility of its teacher network M_s through distillation. This enables EyePAD++ to outperform [26] in almost all of problem settings (Tables 3,4,5,6). Thus, for training an MTL network with distillation, we show that *using a single teacher proficient in both the tasks is better than using two teachers proficient in single tasks* in our disjoint multitasking problem.

5. Conclusion

In this work, we propose two knowledge distillation-based frameworks: EyePAD and EyePAD++ for joint EA and PAD tasks. For evaluating EA, we present a new user-to-user verification protocol and introduce a new metric to jointly measure user-to-user verification and PAD. Our proposed methods outperform the existing baselines (MTL and MTMT) in most of the problem settings. We evaluate our methods using different network backbones and multiple image quality degradation. Additionally, we evaluate our methods to perform eye-to-eye verification with PAD (following previous work). Although we do not use any preprocessing, EyePAD and EyePAD++ outperform the SOTA in PAD and obtain eye-to-eye verification performance that is comparable to SOTA EA algorithms.

Acknowledgement

This work was done when the first author was an intern at Meta. This research is partially supported by a MURI from the Army Research Office under the Grant No. W911NF-17-1-0304. This is part of the collaboration between US DOD, UK MOD and UK Engineering and Physical Research Council (EPSRC) under the Multidisciplinary University Research Initiative.

References

[1] Kevin W Bowyer and Patrick J Flynn. The ND-IRIS-0405 iris image dataset. *arXiv preprint arXiv:1606.04853*, 2016. 5, 8, 11

[2] Cunjian Chen and Arun Ross. A multi-task convolutional neural network for joint iris detection and presentation attack detection. In *2018 IEEE Winter Applications of Computer Vision Workshops (WACVW)*, pages 44–51. IEEE, 2018. 2

[3] Cunjian Chen and Arun Ross. An explainable attention-guided iris presentation attack detector. In *WACV (Workshops)*, pages 97–106, 2021. 2

[4] Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *International Conference on Machine Learning*, pages 794–803. PMLR, 2018. 4

[5] Adam Czajka. Database of iris printouts and its application: Development of liveness detection method for iris recognition. In *2013 18th International Conference on Methods & Models in Automation & Robotics (MMAR)*, pages 28–33. IEEE, 2013. 1

[6] John Daugman. How iris recognition works. In *The essential guide to image processing*, pages 715–739. Elsevier, 2009. 2

[7] John G Daugman. High confidence visual recognition of persons by a test of statistical independence. *IEEE transactions on pattern analysis and machine intelligence*, 15(11):1148–1161, 1993. 1, 2

[8] P Dhar, C Castillo, and R Chellappa. On measuring the iconicity of a face. In *2019 IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 2137–2145. IEEE, 2019. 6

[9] Prithviraj Dhar, Rajat Vikram Singh, Kuan-Chuan Peng, Ziyan Wu, and Rama Chellappa. Learning without memorizing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5138–5146, 2019. 3

[10] Meiling Fang, Naser Damer, Fadi Boutros, Florian Kirchbuchner, and Arjan Kuijper. Deep learning multi-layer fusion for an accurate iris presentation attack detection. In *2020 IEEE 23rd International Conference on Information Fusion (FUSION)*, pages 1–8. IEEE, 2020. 2

[11] Meiling Fang, Naser Damer, Fadi Boutros, Florian Kirchbuchner, and Arjan Kuijper. Iris presentation attack detection by attention-based and deep pixel-wise binary supervision network. In *2021 IEEE International Joint Conference on Biometrics (IJCB)*, pages 1–8. IEEE, 2021. 2, 5, 8

[12] Abhishek Gangwar and Akanksha Joshi. Deepirisnet: Deep iris representation with applications in iris recognition and cross-sensor iris recognition. In *2016 IEEE international conference on image processing (ICIP)*, pages 2301–2305. IEEE, 2016. 2

[13] Fei He, Ye Han, Han Wang, Jinchao Ji, Yuanning Liu, and Zhiqiang Ma. Deep learning architecture for iris recognition based on optimal gabor filters and deep belief network. *Journal of Electronic Imaging*, 26(2):023005, 2017. 2

[14] Lingxiao He, Haiqing Li, Fei Liu, Nianfeng Liu, Zhenan Sun, and Zhaofeng He. Multi-patch convolution neural network for iris liveness detection. In *2016 IEEE 8th International Conference on Biometrics Theory, Applications and Systems (BTAS)*, pages 1–7. IEEE, 2016. 1, 2

[15] Geoffrey Hinton, Oriol Vinyals, and Jeffrey Dean. Distilling the knowledge in a neural network. In *NIPS Deep Learning and Representation Learning Workshop*, 2015. 3

[16] Steven Hoffman, Renu Sharma, and Arun Ross. Convolutional neural networks for iris presentation attack detection: Toward cross-dataset and cross-sensor generalization. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, pages 1620–1628, 2018. 1

[17] Steven Hoffman, Renu Sharma, and Arun Ross. Iris+ ocular: Generalized iris presentation attack detection using multiple convolutional neural networks. In *2019 International Conference on Biometrics (ICB)*, pages 1–8. IEEE, 2019. 2

[18] Yan Hong, Li Niu, Jianfu Zhang, and Liqing Zhang. Beyond without forgetting: Multi-task learning for classification with disjoint datasets. In *2020 IEEE International Conference on Multimedia and Expo (ICME)*, pages 1–6. IEEE, 2020. 2

[19] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1314–1324, 2019. 2, 5, 8, 12

[20] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4700–4708, 2017. 2, 5

[21] Ken Hughes and Kevin W Bowyer. Detection of contact-lens-based iris biometric spoofs using stereo imaging. In *2013 46th Hawaii International Conference on System Sciences*, pages 1763–1772. IEEE, 2013. 1

[22] Dong-Jin Kim, Jinsoo Choi, Tae-Hyun Oh, Youngjin Yoon, and In So Kweon. Disjoint multi-task learning between heterogeneous human-centric tasks. In *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 1699–1708. IEEE, 2018. 2, 3

[23] Gabriela Y Kimura, Diego R Lucio, Alceu S Britto Jr, and David Menotti. Cnn hyperparameter tuning applied to iris liveness detection. *arXiv preprint arXiv:2003.00833*, 2020. 8

[24] Iasonas Kokkinos. Ubernet: Training a universal convolutional neural network for low-, mid-, and high-level vision using diverse datasets and limited memory. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6129–6138, 2017. 3

[25] Andrey Kuehlkamp, Allan Pinto, Anderson Rocha, Kevin W Bowyer, and Adam Czajka. Ensemble of multi-view learning

classifiers for cross-domain iris presentation attack detection. *IEEE Transactions on Information Forensics and Security*, 14(6):1419–1431, 2018. 2, 8

[26] Wei-Hong Li and Hakan Bilen. Knowledge distillation for multi-task learning. In *European Conference on Computer Vision*, pages 163–176. Springer, 2020. 3, 4, 5, 7, 8, 12, 13

[27] Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE transactions on pattern analysis and machine intelligence*, 40(12):2935–2947, 2017. 2, 3, 4

[28] An-An Liu, Yu-Ting Su, Wei-Zhi Nie, and Mohan Kankanhalli. Hierarchical clustering multi-task learning for joint human action grouping and recognition. *IEEE transactions on pattern analysis and machine intelligence*, 39(1):102–114, 2016. 3

[29] Libor Masek et al. *Recognition of human iris patterns for biometric identification*. PhD thesis, Citeseer, 2003. 2, 8

[30] B Maze, J Adams, J A Duncan, N Kalka, T Miller, C Otto, A K Jain, W T Niggel, J Anderson, J Cheney, et al. IARPA janus benchmark-c: Face dataset and protocol. In *2018 International Conference on Biometrics (ICB)*, pages 158–165. IEEE, 2018. 6

[31] David Menotti, Giovani Chiachia, Allan Pinto, William Robson Schwartz, Helio Pedrini, Alexandre Xavier Falcao, and Anderson Rocha. Deep representations for iris, face, and fingerprint spoofing detection. *IEEE Transactions on Information Forensics and Security*, 10(4):864–879, 2015. 2

[32] Kien Nguyen, Clinton Fookes, Arun Ross, and Sridha Sridharan. Iris recognition with off-the-shelf cnn features: A deep learning perspective. *IEEE Access*, 6:18848–18855, 2017. 2

[33] Federico Pala and Bir Bhanu. Iris liveness detection by relative distance comparisons. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, pages 162–169, 2017. 2

[34] P Jonathon Phillips, W Todd Scruggs, Alice J O'Toole, Patrick J Flynn, Kevin W Bowyer, Cathy L Schott, and Matthew Sharpe. Frvt 2006 and ice 2006 large-scale experimental results. *IEEE transactions on pattern analysis and machine intelligence*, 32(5):831–846, 2009. 5

[35] Ramachandra Raghavendra, Kiran B Raja, and Christoph Busch. Contlensnet: Robust iris contact lens detection using deep convolutional neural networks. In *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 1160–1167. IEEE, 2017. 2

[36] R Ranjan, A Bansal, J Zheng, H Xu, J Gleason, B Lu, A Nanduri, J-C Chen, C D Castillo, and R Chellappa. A fast and accurate system for face detection, identification, and verification. *IEEE Transactions on Biometrics, Behavior, and Identity Science*, 1(2):82–96, 2019. 6

[37] R Ranjan, S Sankaranarayanan, C D Castillo, and R Chellappa. An all-in-one convolutional neural network for face analysis. In *2017 12th IEEE International Conference on Automatic Face & Gesture Recognition (FG 2017)*, pages 17–24. IEEE, 2017. 2, 3, 4

[38] Christian Rathgeb, Andreas Uhl, Peter Wild, and Heinz Hofbauer. Design decisions for an iris recognition sdk. In *Handbook of iris recognition*, pages 359–396. Springer, 2016. 1, 2

[39] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. iCaRL: Incremental classifier and representation learning. In *Proc. CVPR*, 2017. 3

[40] Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. Fitnets: Hints for thin deep nets. *ICLR 2015*, 2015. 3, 4

[41] Renu Sharma and Arun Ross. D-netpad: An explainable and interpretable iris presentation attack detector. In *2020 IEEE International Joint Conference on Biometrics (IJCB)*, pages 1–10. IEEE, 2020. 1, 2, 5, 8

[42] Zhenan Sun and Tieniu Tan. Ordinal measures for iris recognition. *IEEE Transactions on pattern analysis and machine intelligence*, 31(12):2211–2226, 2008. 2, 8

[43] Xingqiang Tang, Jiangtao Xie, and Peihua Li. Deep convolutional features for iris recognition. In *Chinese conference on biometric recognition*, pages 391–400. Springer, 2017. 2

[44] Jingdong Wang, Ke Sun, Tianheng Cheng, Borui Jiang, Chaorui Deng, Yang Zhao, Dong Liu, Yadong Mu, Mingkui Tan, Xinggang Wang, et al. Deep high-resolution representation learning for visual recognition. *IEEE transactions on pattern analysis and machine intelligence*, 2020. 2, 5, 8, 11

[45] Kuo Wang and Ajay Kumar. Toward more accurate iris recognition using dilated residual features. *IEEE Transactions on Information Forensics and Security*, 14(12):3233–3245, 2019. 1, 2, 3, 4, 5, 8

[46] Daksha Yadav, Naman Kohli, Akshay Agarwal, Mayank Vatsa, Richa Singh, and Afzel Noore. Fusion of handcrafted and deep learning features for large-scale multiple iris presentation attack detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, pages 572–579, 2018. 2

[47] Daksha Yadav, Naman Kohli, Mayank Vatsa, Richa Singh, and Afzel Noore. Detecting textured contact lens in uncontrolled environment using densepad. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 0–0, 2019. 1, 2, 5

[48] David Yambay, Benedict Becker, Naman Kohli, Daksha Yadav, Adam Czajka, Kevin W Bowyer, Stephanie Schuckers, Richa Singh, Mayank Vatsa, Afzel Noore, et al. Livdet iris 2017—iris liveness detection competition 2017. In *2017 IEEE International Joint Conference on Biometrics (IJCB)*, pages 733–741. IEEE, 2017. 5, 6, 8

[49] David Yambay, James S. Doyle, Kevin W. Bowyer, Adam Czajka, and Stephanie Schuckers. Livdet-iris 2013 - iris liveness detection competition 2013. In *IEEE International Joint Conference on Biometrics*, pages 1–8, 2014. 5

[50] David Yambay, Brian Walczak, Stephanie Schuckers, and Adam Czajka. Livdet-iris 2015 - iris liveness detection competition 2015. In *2017 IEEE International Conference on Identity, Security and Behavior Analysis (ISBA)*, pages 1–6, 2017. 5

[51] Kai Yang, Zihao Xu, and Jingjing Fei. Dualsanet: Dual spatial attention network for iris recognition. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 889–897, 2021. 2

[52] Sergey Zagoruyko and Nikos Komodakis. Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer. In *ICLR*, 2017. 3

[53] Zijing Zhao and Ajay Kumar. Towards more accurate iris

recognition using deeply learned spatially corresponding features. In *Proceedings of the IEEE international conference on computer vision*, pages 3809–3818, 2017. 1, 2, 5, 8

Supplementary material

In this supplementary material, we provide the following information:
Section A1: Train and test split for user-to-user verification.
Section A2: Hyperparameters for EyePAD and EyePAD++.
Section A3: Detailed results with HRnet and MobilenetV3 backbones.
Section A4: Ablation experiments for λ_1 (EyePAD).
Section A5: Hyperparameters for baseline methods.

A1. Train and test splits for ND-Iris-0405 dataset [1]

In section 4.2 of the main paper, we mention that we randomly split the users into two subsets: U_{train} (for training) and U_{test} (for evaluation). All the images for users in the training split are used for training. Here, we provide the train and test split to enable researchers reproduce our protocol.

Train user IDs: '04200' '04203' '04214' '04233' '04239' '04261' '04265' '04267' '04284' '04286' '04288' '04302' '04309' '04313' '04320' '04327' '04336' '04339' '04349' '04351' '04361' '04370' '04378' '04379' '04382' '04387' '04394' '04395' '04397' '04400' '04407' '04408' '04409' '04418' '04419' '04429' '04430' '04434' '04435' '04436' '04440' '04446' '04447' '04453' '04460' '04471' '04472' '04475' '04476' '04477' '04479' '04481' '04482' '04485' '04495' '04496' '04502' '04504' '04505' '04506' '04511' '04512' '04514' '04530' '04535' '04542' '04553' '04560' '04575' '04577' '04578' '04581' '04587' '04588' '04589' '04593' '04596' '04597' '04598' '04603' '04605' '04609' '04613' '04615' '04622' '04626' '04628' '04629' '04632' '04633' '04634' '04644' '04647' '04653' '04670' '04684' '04687' '04691' '04692' '04695' '04699' '04701' '04702' '04703' '04712' '04715' '04716' '04720' '04721' '04725' '04729' '04734' '04736' '04737' '04738' '04742' '04744' '04745' '04747' '04748' '04751' '04756' '04757' '04758' '04763' '04765' '04768' '04772' '04773' '04774' '04776' '04777' '04778' '04782' '04783' '04785' '04787' '04790' '04792' '04794' '04797' '04801' '04802' '04803' '04813' '04815' '04816' '04818' '04831' '04832' '04839' '04840' '04841' '04843' '04846' '04847' '04850' '04854' '04857' '04858' '04859' '04861' '04863' '04864' '04866' '04867' '04869' '04870' '04871' '04872' '04873' '04876' '04877' '04878' '04879' '04880' '04882' '04883' '04884' '04886' '04888' '04890' '04891' '04892' '04894' '04897' '04898' '04899' '04901' '04905' '04908' '04909' '04910' '04911' '04912' '04914' '04915' '04919' '04920' '04922' '04923' '04928' '04930' '04931' '04932' '04934'

Test user IDs:'02463' '04201' '04202' '04213' '04217' '04221' '04225' '04273' '04285' '04297' '04300' '04301' '04311' '04312' '04314' '04319' '04322' '04324' '04334' '04338' '04341' '04343' '04344' '04347' '04350' '04372' '04385' '04388' '04404' '04423' '04427' '04444' '04449' '04451' '04456' '04459' '04461' '04463' '04470' '04473' '04488' '04493' '04507' '04509' '04513' '04519' '04531' '04537' '04556' '04557' '04569' '04580' '04585' '04595' '04600' '04612' '04621' '04631' '04641' '04662' '04664' '04667' '04673' '04675' '04681' '04682' '04683' '04689' '04693' '04697' '04705' '04708' '04709' '04711' '04714' '04719' '04722' '04724' '04726' '04727' '04728' '04730' '04731' '04732' '04733' '04743' '04746' '04749' '04754' '04760' '04762' '04767' '04770' '04775' '04784' '04786' '04796' '04798' '04806' '04810' '04811' '04812' '04821' '04822' '04823' '04827' '04829' '04830' '04833' '04838' '04842' '04848' '04849' '04851' '04853' '04855' '04856' '04860' '04862' '04865' '04868' '04874' '04875' '04881' '04885' '04887' '04889' '04893' '04895' '04896' '04900' '04902' '04903' '04904' '04906' '04907' '04913' '04916' '04917' '04918' '04921' '04924' '04925' '04926' '04927' '04929' '04933' '04935' '04936'

It is also mentioned in the main paper that the images for the test users are then randomly divided into query and gallery sets. We provide the images in the query and gallery sets in `query_set.txt` and `gallery_set.txt`, respectively. These files are provided here. We also provide a readme file (`README.txt`) for the readers' convenience, wherein we provide information about the left/right labels.

A2. Training details for EyePAD, EyePAD++

We train all the models in our work with a batch size of 64 in our experiments, for 100 epochs. We use data augmentation such as random horizontal flip, random rotation (30 degrees) and random jitter. The detailed hyperparameter information for training models for user-to-user verification with PAD is provided in Table A3.
While training Densenet121 network for eye-to-eye verification with PAD, we use $\lambda_1 = 2.0$ (for EyePAD) and $\lambda_2 = 2.0$ (for EyePAD++). All the other parameters used in this experiment are same as those mentioned in the first row of Table A3.

A3. Detailed results

A3.1. EA and PAD with HRnet64

In Table A1, we provide the full version of Table 4 from the main paper, where we present results with HRnet64 [44]. Here, we report the user-to-user verification performance

	User-to-user verification results on ND-Iris-0405 (EA)												PAD results on CU-LivDet			
	1 Query 1 Gallery				1 Query 2 Gallery				1 Query 5 Gallery							
Method	OFRR(↓)	10^{-4}	10^{-3}	10^{-2}	OFRR(↓)	10^{-4}	10^{-3}	10^{-2}	OFRR(↓)	10^{-4}	10^{-3}	10^{-2}	TDR(↑)	APCER	BPCER	HTER(↓)
EA only	-	0.861	0.950	0.990	-	0.875	0.961	0.994	-	0.919	0.983	0.996	-	-	-	-
PAD only	-	-	-	-	-	-	-	-	-	-	-	-	0.962	0.051	0.00	0.026
MTL	0.209	0.682	0.819	0.949	0.156	0.734	0.866	0.962	0.113	0.815	0.930	0.977	0.921	0.053	0.005	0.029
MTMT [26]	0.132	0.777	0.881	0.970	0.091	0.840	0.917	0.971	0.068	0.891	0.945	0.984	0.959	0.033	0.001	0.017
EyePAD	0.094	0.804	0.909	0.985	0.060	0.864	0.942	0.993	0.034	0.898	0.968	0.995	0.934	0.044	0.014	0.029
EyePAD++	**0.062**	0.865	0.942	0.996	**0.048**	0.898	0.959	0.993	**0.031**	0.926	0.976	0.997	0.915	0.041	0.021	0.031
EA only	-	0.829	0.913	0.978	-	0.838	0.944	0.988	-	0.911	0.968	0.992	-	-	-	-
PAD only	-	-	-	-	-	-	-	-	-	-	-	-	0.801	0.089	0.026	0.058
MTL	0.430	0.503	0.650	0.851	0.377	0.509	0.719	0.868	0.293	0.638	0.801	0.924	0.737	0.040	0.331	0.186
MTMT [26]	0.188	0.768	0.898	0.973	0.165	0.846	0.932	0.990	0.129	0.904	0.963	0.994	0.646	0.125	0.046	0.086
EyePAD	0.180	0.805	0.897	0.970	**0.137**	0.877	0.932	0.989	0.132	0.902	0.960	0.993	0.766	0.115	0.008	0.062
EyePAD++	**0.174**	0.830	0.911	0.983	0.158	0.869	0.950	0.988	**0.118**	0.915	0.971	0.989	0.655	0.109	0.025	0.067
EA only	-	0.733	0.893	0.984	-	0.749	0.924	0.990	-	0.837	0.952	0.992	-	-	-	-
PAD only	-	-	-	-	-	-	-	-	-	-	-	-	0.942	0.049	0.008	0.029
MTL	0.338	0.451	0.678	0.856	0.279	0.481	0.737	0.908	0.236	0.587	0.787	0.940	0.899	0.085	0.004	0.045
MTMT [26]	0.184	0.675	0.846	0.944	0.168	0.721	0.862	0.961	0.114	0.824	0.916	0.975	0.894	0.048	0.020	0.034
EyePAD	0.161	0.656	0.848	0.964	0.125	0.718	0.886	0.984	**0.091**	0.800	0.916	0.986	0.914	0.060	0.006	0.033
EyePAD++	**0.157**	0.729	0.869	0.976	**0.114**	0.781	0.916	0.988	0.093	0.840	0.937	0.989	0.887	0.061	0.010	0.036

Table A1. EA and PAD with HRnet64 trained and evaluated on **(top)** original, **(middle)** blurred, **(bottom)** Noisy data: For user-to-user verification, we report TAR@FAR=$10^{-4}, 10^{-3}, 10^{-2}$. For PAD, we report TDR@FDR=0.002 and APCER, BPCER, HTER. OFRR jointly measures EA and PAD performance on ND-Iris-0405. *EyePAD++ obtains the lowest OFRR in most scenarios*. **Bold**: Best, Underlined: Second best.

	User-to-user verification results on ND-Iris-0405 (EA)												PAD results on CU-LivDet			
	1 Query 1 Gallery				1 Query 2 Gallery				1 Query 5 Gallery							
Method	OFRR(↓)	10^{-4}	10^{-3}	10^{-2}	OFRR(↓)	10^{-4}	10^{-3}	10^{-2}	OFRR(↓)	10^{-4}	10^{-3}	10^{-2}	TDR(↑)	APCER	BPCER	HTER(↓)
EA only	-	0.824	0.923	0.989	-	0.863	0.943	0.987	-	0.898	0.952	0.995	-	-	-	-
PAD only	-	-	-	-	-	-	-	-	-	-	-	-	0.925	0.048	0.010	0.029
MTL	0.180	0.739	0.856	0.958	0.142	0.792	0.895	0.970	0.110	0.872	0.933	0.985	0.884	0.025	0.053	0.039
MTMT [26]	0.193	0.751	0.871	0.973	0.144	0.817	0.917	0.977	0.126	0.859	0.933	0.987	0.793	0.073	0.011	0.042
EyePAD	0.160	0.769	0.893	0.972	0.142	0.838	0.919	0.985	0.114	0.887	0.947	0.991	0.859	0.046	0.034	0.040
EyePAD++	**0.140**	0.819	0.904	0.981	**0.117**	0.863	0.934	0.992	**0.085**	0.901	0.962	0.990	0.883	0.041	0.022	0.032
EA only	-	0.889	0.953	0.993	-	0.853	0.929	0.992	-	0.846	0.921	0.989	-	-	-	-
PAD only	-	-	-	-	-	-	-	-	-	-	-	-	0.581	0.155	0.078	0.117
MTL	0.564	0.585	0.734	0.871	0.533	0.619	0.798	0.927	0.483	0.744	0.855	0.942	0.556	0.133	0.122	0.128
MTMT [26]	0.524	0.642	0.819	0.939	0.477	0.726	0.905	0.965	0.464	0.769	0.913	0.963	0.502	0.227	0.046	0.137
EyePAD	0.562	0.537	0.715	0.915	0.486	0.618	0.810	0.952	0.447	0.711	0.866	0.956	0.589	0.144	0.097	0.121
EyePAD++	**0.385**	0.768	0.874	0.956	**0.372**	0.792	0.913	0.970	**0.332**	0.861	0.938	0.983	0.552	0.092	0.173	0.133
EA only	-	0.756	0.872	0.977	-	0.795	0.912	0.982	-	0.817	0.944	0.985	-	-	-	-
PAD only	-	-	-	-	-	-	-	-	-	-	-	-	0.831	0.079	0.003	0.041
MTL	0.263	0.643	0.799	0.949	0.217	0.682	0.870	0.976	0.173	0.761	0.908	0.974	0.762	0.062	0.065	0.064
MTMT [26]	0.285	0.549	0.766	0.928	0.185	0.705	0.868	0.968	0.162	0.777	0.906	0.977	0.730	0.049	0.069	0.059
EyePAD	0.262	0.722	0.847	0.958	0.227	0.779	0.880	0.974	0.209	0.801	0.927	0.980	0.712	0.083	0.047	0.065
EyePAD++	**0.254**	0.660	0.786	0.947	**0.180**	0.733	0.868	0.975	**0.137**	0.811	0.912	0.990	0.730	0.033	0.126	0.080

Table A2. EA and PAD with MobilenetV3 trained and evaluated on **(top)** original, **(middle)** blurred, **(bottom)** Noisy data: For user-to-user verification, we report TAR@FAR=$10^{-4}, 10^{-3}, 10^{-2}$. For PAD, we report TDR@FDR=0.002 and APCER, BPCER, HTER. OFRR jointly measures EA and PAD performance on ND-Iris-0405. *EyePAD++ obtains the lowest OFRR*. **Bold**: Best, Underlined: Second best.

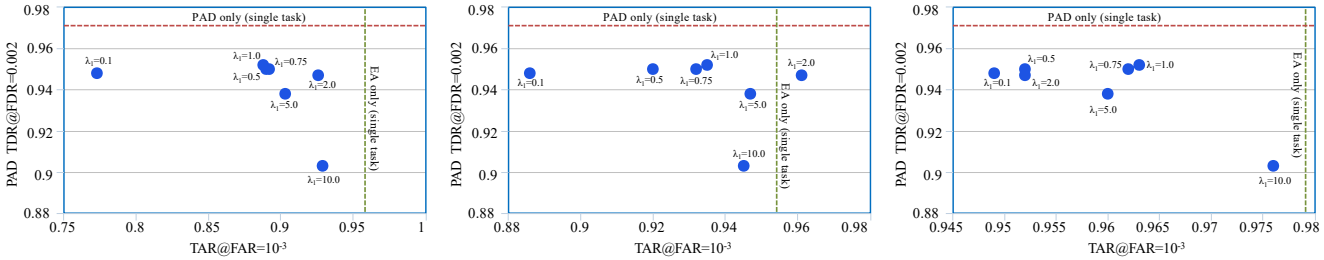
with $K = 1, 2, 5$ gallery pairs. In most of the scenarios, *EyePAD++ outperforms the existing baselines in terms of the OFRR score*.

A3.2. EA and PAD with MobilenetV3

In Table A2, we provide the full version of Table 5 from the main paper, where we present results with MobilenetV3 [19]. Here, we report the user-to-user verification performance with $K = 1, 2, 5$ gallery pairs. *EyePAD++ outperforms the existing baselines in terms of the OFRR score*, in all the problem settings.

A4. Ablation study: Effect of λ_1 in EyePAD

The hyperparameter λ_1 is used to weight the feature-level distillation loss L_{dis} (Eq 2 of the main paper). L_{dis} is used to preserve the EA information, while student M_s (initialized with M_t) is trained for PAD. Using Densenet121 and the original (clean) EA and PAD datasets, we analyze the effect of λ_1 on user-to-user verification perfor-



(a) 1 query 1 gallery (b) 1 query 2 gallery (c) 1 query 5 gallery

Figure A1. PAD performance (TDR@FDR=0.002) v/s User-to-user verification performance (TAR@FAR=10^{-3}) obtained by the EyePAD student network, for different values of λ_1.

Backbone	Dataset	λ_1	λ_2	Optimizer	LR	γ	Decay after
Densenet121	Original	2.0	0.75	Adam	10^{-4}	0.5	12
Densenet121	Blurred	1.0	2.0	Adam	10^{-4}	0.5	12
Densenet121	Noisy	1.0	2.0	Adam	10^{-4}	0.5	12
HRnet64	Original	2.0	2.0	Adam	10^{-4}	0.5	12
HRnet64	Blurred	5.0	2.0	Adam	10^{-4}	0.5	12
HRnet64	Noisy	2.0	5.0	Adam	10^{-4}	0.5	12
MobilenetV3	Original	1.0	0.75	SGD	10^{-1}	0.1	15
MobilenetV3	Blurred	1.0	0.75	SGD	10^{-1}	0.1	15
MobilenetV3	Noisy	5.0	2.0	SGD	10^{-1}	0.1	15

Table A3. Hyperparameter information for EyePAD and EyePAD++.

Backbone	Dataset	λ_{auth}	λ_{pad}	Optimizer	LR	γ	Decay after
Densenet121	Original	1.0	1.0	Adam	10^{-4}	0.5	12
Densenet121	Blurred	0.75	0.75	Adam	10^{-4}	0.5	12
Densenet121	Noisy	0.5	0.75	Adam	10^{-4}	0.5	12
HRnet64	Original	1.0	1.0	Adam	10^{-4}	0.5	12
HRnet64	Blurred	0.75	0.75	Adam	10^{-4}	0.5	12
HRnet64	Noisy	2.0	2.0	Adam	10^{-4}	0.5	12
MobilenetV3	Original	1.0	2.0	SGD	10^{-1}	0.1	15
MobilenetV3	Blurred	1.0	1.0	SGD	10^{-1}	0.1	15
MobilenetV3	Noisy	0.1	0.1	SGD	10^{-1}	0.1	15

Table A4. Hyperparameter information for MTMT [26].

mance and PAD performance. We perform experiments with $\lambda_1 = [0.1, 0.5, 0.75, 1.0, 2.0, 5.0, 10.0]$ and present the corresponding results in Fig. A1. We find that in general, when λ_1 increases, the user to user verification performance improves and the PAD performance gets degraded. This is expected because a higher value for λ_1 enforces M_s to preserve authentication and restrics it from learning PAD-specific features. However, we do not find any such trend with respect to parameter λ_2.

A5. Training details for baselines

For training the MTL baseline for user-to-user or eye-to-eye verification with PAD, we use the same parameter values mentioned in Table A3, except for λ_1, λ_2. The hyperparameters used for training MTMT [26] are provided in Table A4. While training MTMT [26] for eye-to-eye verification with PAD (using Densenet121 and the original dataset), we use $\lambda_{auth} = 1.0$ and $\lambda_{pad} = 1.0$. The rest of the hyperparameters are same as those mentioned in the first row of Table A4.